

Principios S O L I D

Cada letra define una regla de diseño para el desarrollo ordenado con clases


SOLIDO

La idea detrás de estos principios, es que al aplicarlos en conjunto para programar y diseñar nuestro software, es más probable que generemos un código ordenado, mantenible y escalable a largo plazo.

No son reglas inmutables, fijas, ni obligatorias. Se deben aplicar con criterio, pero en la mayoría de los casos, **su inclusión es mejor que su exclusión**



Single Responsibility Principle

(Principio de responsabilidad unica)

Cada clase debe tener una única responsabilidad (para la cual fue creada) y debido a esto se limita a una sola razón para cambiar

> Esto no significa que la clase va a tener 1 solo método o que va a ser chica, sino que las responsabilidades deben estar bien divididas.

SRP

Ejemplo

```php
class Usuario
{
    public function guardar()
    {
        // Guardar usuario en la base de datos
    }

    public function enviarEmail()
    {
        // Enviar email
    }

    public function generarReporte()
    {
        // Generar reporte
    }
}
```

La clase anterior es un ejemplo erróneo ¿Cual es el problema con la misma?

SRP

Aplicando SRP se vería de la siguiente manera:

```php
class Usuario
{
    // Información y comportamiento propio del usuario
}
```

```php
class UsuarioRepository
{
    public function guardar(Usuario $usuario)
    {
        // Persistencia
    }
}
```

```php
class EmailService
{
    public function enviar(string $email, string $mensaje)
    {
        // Envío de emails
    }
}
```

```php
class ReportService
{
    public function generar(Usuario $usuario)
    {
        // Generación de reportes
    }
}
```

Donde cada clase separa sus responsabilidades, si tenemos que cambiar algo en la lógica de envío de mails, vamos directamente a **“EmailService”** tocamos el método ahi y no afectamos a las demás clases



OCP

(Principio de abierto/cerrado)

Una clase debería estar abierta para extenderla, pero cerrada para modificarla.

> La idea es que podamos agregar nuevos comportamientos sin tener que modificar constantemente el código existente.

S **O** L I D

OCP

```php
class NotificationService
{
    public function send(string $type, string $message)
    {
        if ($type === 'email') {
            // enviar email
        }

        if ($type === 'sms') {
            // enviar SMS
        }
    }
}
```

¿Cual es el problema en este código?

Para cada nuevo tipo de notificación, requerimos ir a la clase específica y modificarla, no solo que se vuelve engorroso si el código es largo, sino que además **corremos el riesgo de romper algo en la lógica preexistente de la clase**, causando que funcionalidades relacionadas pero no modificadas fallen o cambien.

OCP

Aplicando OCP

Podemos definir una abstracción:

```php
interface Notification
{
    public function send(string $message): void;
}
```

Y diferentes implementaciones:

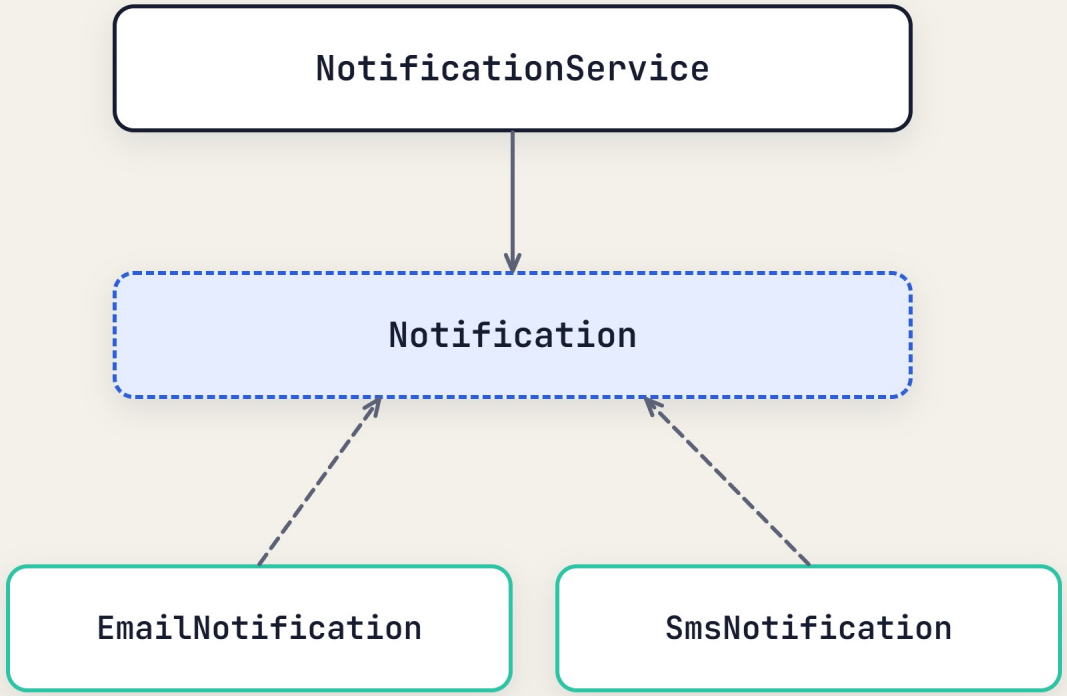
```php
class EmailNotification implements Notification
{
    public function send(string $message): void
    {
        // enviar email
    }
}
```

```php
class SmsNotification implements Notification
{
    public function send(string $message): void
    {
        // enviar SMS
    }
}
```

OCP

El servicio trabaja con la abstracción:

```php
class NotificationService
{
    public function send(
        Notification $notification,
        string $message
    ): void {
        $notification->send($message);
    }
}
```





LSP

(Principio de sustitución de Liskov)

Una clase hija debe poder utilizarse en cualquier lugar donde se espera su clase padre, sin romper el comportamiento esperado del programa.

> Esto significa que, si nuestra clase hija, hereda un comportamiento de una clase padre, el mismo debe tener un sentido para la clase hija y no quedar huérfano o inutilizable.

LSP

Por ejemplo:

```php
class Ave
{
    public function volar(): void
    {
        echo "Volando";
    }
}
```

```php
class Aguila extends Ave
{
}
```

```php
class Pinguino extends Ave
{
}
```

¿Cual es el problema de LSP de la clase Pingüino?

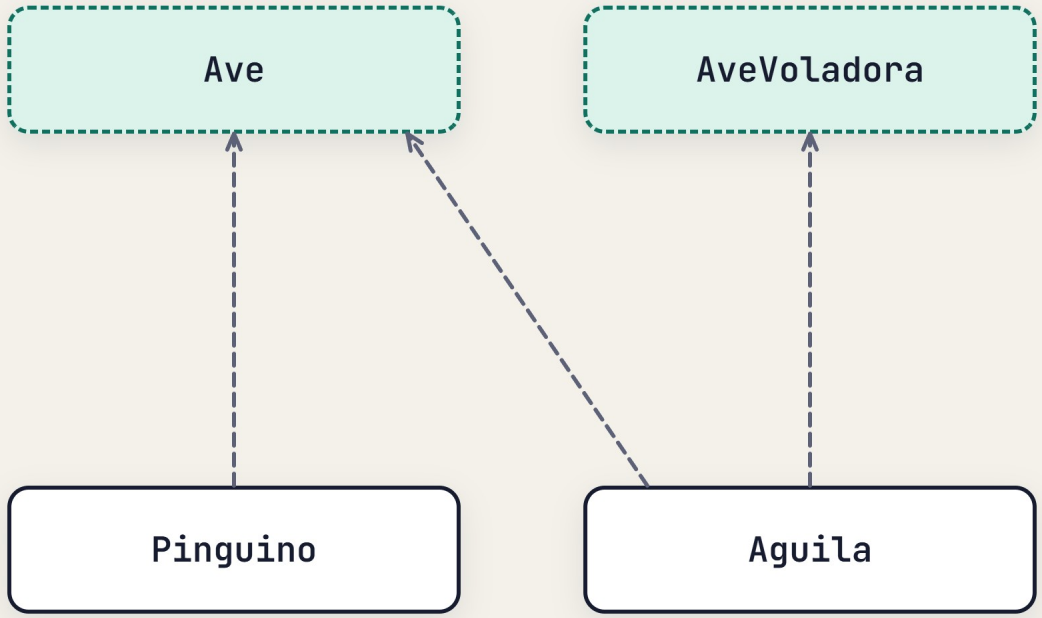
LSP

Ejemplo con LSP

Separamos las capacidades:

```php
interface Ave
{
    public function comer(): void;
}
```

```php
interface AveVoladora
{
    public function volar(): void;
}
```



LSP

Entonces:

```php
class Aguila implements Ave, AveVoladora
{
    public function comer(): void
    {
    }

    public function volar(): void
    {
        echo "Volando";
    }
}
```

```php
class Pinguino implements Ave
{
    public function comer(): void
    {
    }
}
```



ISP

(Principio de segregación de la interfaz)

Cada interfaz debe ser pequeña y específica, limitándose a las tareas para las que fue creada y evitando implantar métodos no utilizados por las clases que las heredan

ISP

Ejemplo:

```php
interface NotificationService
{
    public function sendEmail(string $message);
    public function sendSMS(string $message);
    public function sendPush(string $message);
}
```

¿Cuál es el problema claro?

```php
class EmailNotification implements NotificationService
{
    public function sendEmail(string $message)
    {
        // Enviar email
    }

    public function sendSMS(string $message)
    {
        // No corresponde
    }

    public function sendPush(string $message)
    {
        // No corresponde
    }
}
```

ISP

Con ISP

```php
interface EmailSender
{
    public function sendEmail(string $message);
}
```

```php
interface SMSSender
{
    public function sendSMS(string $message);
}
```

```php
interface PushSender
{
    public function sendPush(string $message);
}
```

```php
class EmailNotification implements EmailSender
{
    public function sendEmail(string $message)
    {
        // Enviar email
    }
}
```



DIP

(Principio de inversión de dependencias)

El principio DIP postula que cada clase y métodos deben depender de abstracciones, no de implementaciones

S O L I D

DIP

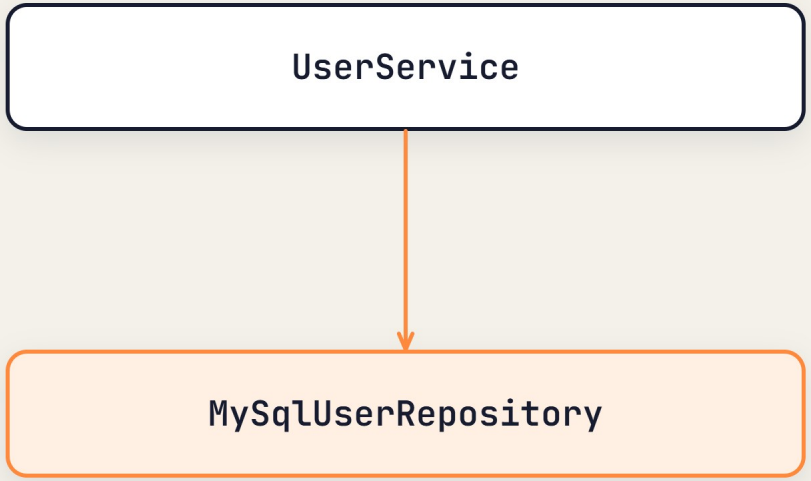
Ejemplo sin DIP

Supongamos un servicio que guarda usuarios:

```php
class UserService
{
    private MySqlUserRepository $repository;

    public function __construct()
    {
        $this->repository = new MySqlUserRepository();
    }

    public function crear(User $user)
    {
        $this->repository->save($user);
    }
}
```



DIP

Aplicando DIP

Creamos una abstracción:

```php
interface UserRepository
{
    public function save(User $user): void;
}
```

Ahora UserService depende de la interfaz:

```php
class UserService
{
    public function __construct(
        private UserRepository $repository
    ) {}

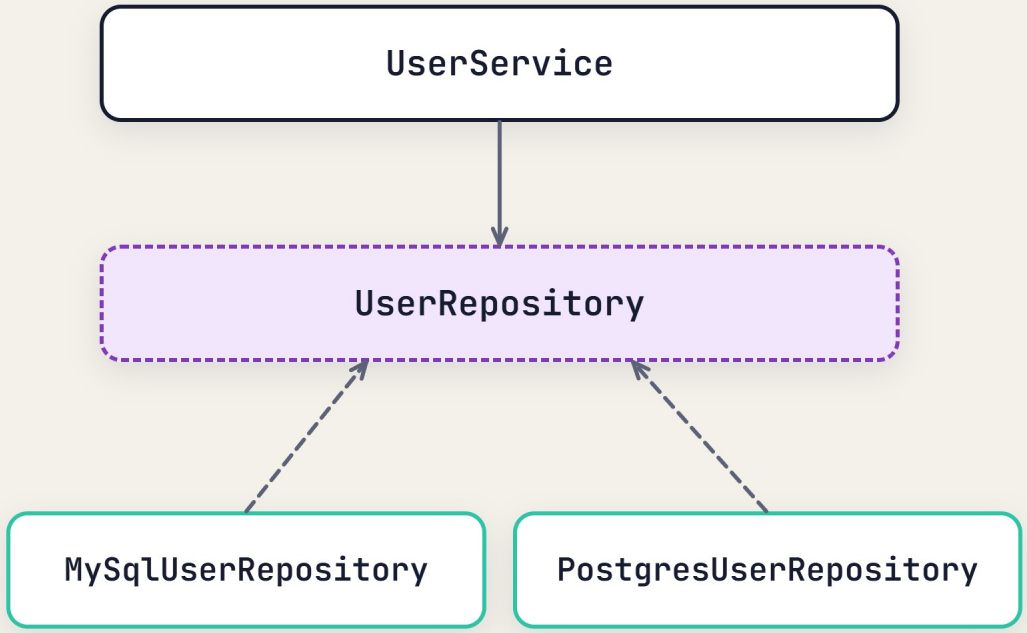
    public function crear(User $user)
    {
        $this->repository->save($user);
    }
}
```

DIP

Y las implementaciones dependen de esa abstracción:

```php
class MySqlUserRepository implements UserRepository
{
    public function save(User $user): void
    {
        // MySQL
    }
}
```

```php
class PostgresUserRepository implements UserRepository
{
    public function save(User $user): void
    {
        // PostgreSQL
    }
}
```



Componente	¿Qué hace?	Pregunta que responde
Controller	Recibe la petición y coordina la respuesta	¿Qué hago con esta petición?
Service	Contiene/coordina lógica de negocio	¿Qué debe hacer el sistema?
Repository	Se ocupa del acceso a datos	¿Cómo obtengo/guardo los datos?
DTO	Transporta datos entre capas	¿Qué datos necesito transportar?
Entity / Model	Representa una entidad del dominio	¿Qué es esta cosa dentro del sistema?
Middleware	Intercepta peticiones antes/después del Controller	¿Hay algo que debo comprobar antes?
Validator	Valida datos de entrada o reglas específicas	¿Estos datos son válidos?
Mapper	Convierte un objeto/estructura en otro	¿Cómo transformo estos datos?
Factory	Se encarga de crear objetos	¿Qué objeto tengo que crear?
View	Presenta información al usuario	¿Cómo muestro el resultado?

COMPONENTE

1. Controller

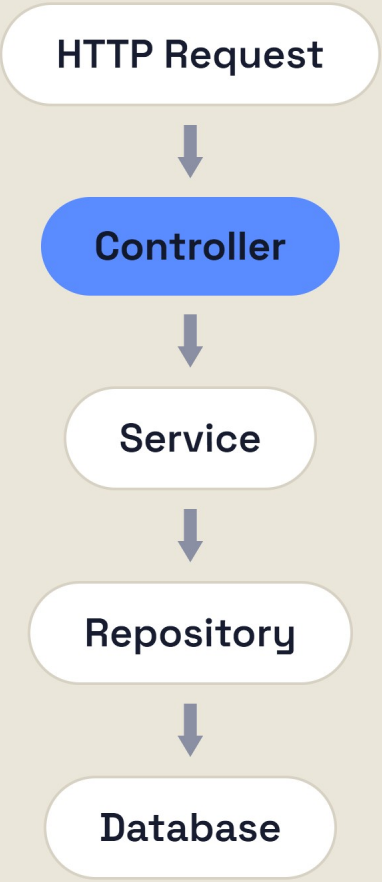
Recibe una entrada (normalmente peticion http), obtiene los datos necesarios, llama a la lógica correspondiente y devuelve una respuesta.

```php
class UserController
{
    public function create(Request $request)
    {
        if (!filter_var($request->email, FILTER_VALIDATE_EMAIL)) {
            return response()->json([
                'error' => 'Email inválido'
            ], 400);
        }

        $user = $this->userService->create($request->data);

        return response()->json($user);
    }
}
```

Normalmente:



COMPONENTE

2. Service

Contiene o coordina lógica de negocio.

Ejemplos:

- registrar un usuario
- procesar una autorización
- realizar una compra
- calcular un precio
- coordinar varios repositorios y servicios

Service = "¿Qué tiene que hacer el sistema?"

```php
class UserService
{
    public function __construct(
        private UserRepository $repository,
        private PasswordHasher $passwordHasher
    ) {}

    public function register(CreateUserDTO $dto)
    {
        $password = $this->passwordHasher->hash($dto->password);

        $user = new User(
            $dto->name,
            $dto->email,
            $password
        );

        return $this->repository->save($user);
    }
}
```

COMPONENTE

3. Repository

Se encarga de persistir y recuperar información.

```php
class UserRepository
{
    public function findById(int $id): ?User
    {
        // SELECT ...
    }

    public function save(User $user): void
    {
        // INSERT / UPDATE ...
    }
}
```



El Service no necesita saber si los datos vienen de:

- PostgreSQL
- MySQL
- API
- archivo
- etc.

COMPONENTE

4. DTO — Data Transfer Object

Un DTO es un objeto utilizado para transportar datos entre partes de la aplicación.

COMPONENTE

Por ejemplo, recibimos de una api externa:

```
{
  "id": 58321,
  "created_at": "2026-09-23T13:42:10Z",
  "status": "active",
  "user": {
    "id": 123,
    "name": "Juan Pérez",
    "email": "juan@gmail.com",
    "phone": "1122334455",
    "address": {
      "street": "Av. Siempre Viva",
      "number": 742,
      "city": "Buenos Aires"
    }
  },
  "company": {
    "id": 55,
    "name": "Empresa SA",
    "country": "Argentina"
  },
  "metadata": {
    "source": "web",
    "ip": "192.168.1.20",
    "device": "Chrome"
  },
  "notifications": [],
  "permissions": [
    "read",
    "write"
  ]
}
```

COMPONENTE

Pero nosotros solo necesitamos nombre y email para crear el usuario.

Armamos el DTO:

```php
class CreateUserDTO
{
    public function __construct(
        public string $name,
        public string $email
    ) {}
}
```

La idea es evitar pasar estructuras arbitrarias por toda la aplicación.

Entonces en nuestro controller hacemos lo siguiente

```php
$data = $request->json();

$dto = new CreateUserDTO(
    name: $data['user']['name'],
    email: $data['user']['email']
);
```

▷ Ahora, a través del service y todas las clases solo vamos a estar moviendo el objeto con los datos que necesitamos

COMPONENTE

5. Entity / Model

Representa una entidad del dominio.

Por ejemplo:

```php
class User
{
    private string $name;
    private string $email;

    public function changeEmail(string $email)
    {
        // ...
    }
}
```

Puede contener estado y comportamiento propio de esa entidad.

Ejemplos:

`User` `Patient` `Invoice` `Product` `Order` `Authorization`

Entity = "¿Qué representa esta cosa dentro del negocio?"